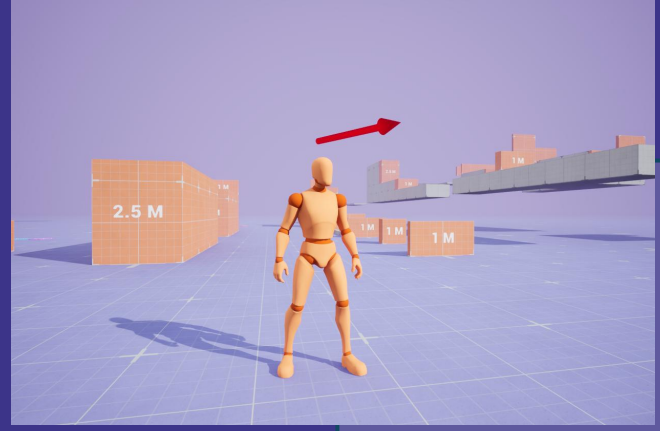
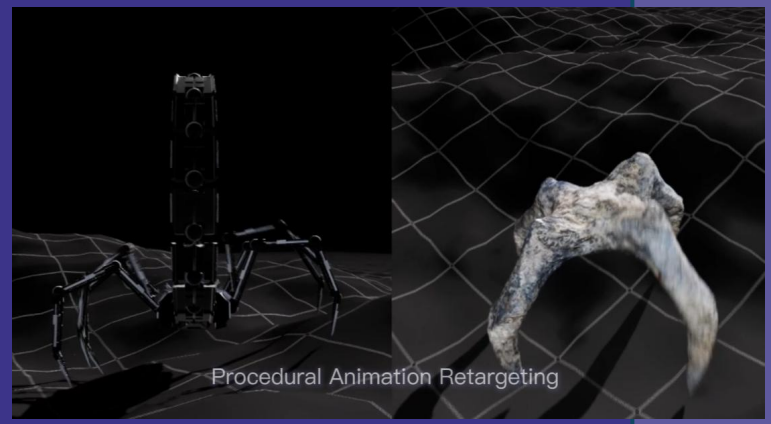
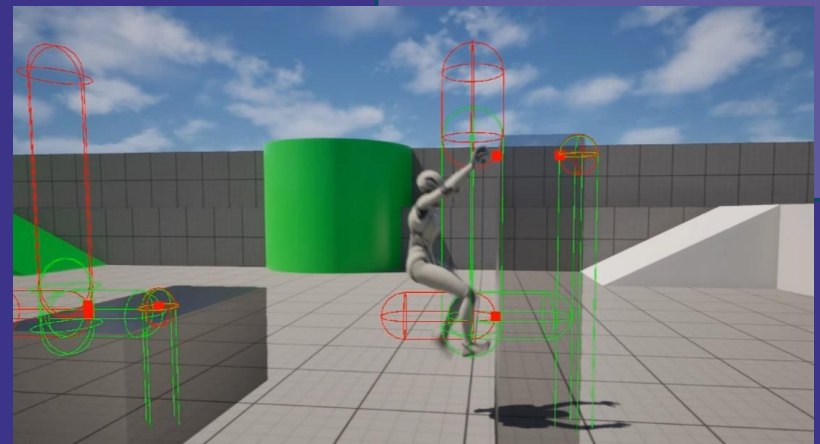
于卓平

动画TA作品集 / Animation TA Portfolio

UE5 · Unity · Blender







Mail: 784349997@qq.com TEL/Wechat: 13338747156

关于我 / About Me

同济大学 · 设计硕士在读 | 本科华侨大学 · 数字媒体技术背景 | 2年动画设计工作经验
聚焦实时角色动画、Animation Blueprint、Control Rig 与程序化动画
熟悉 UE5 / Unity / Blender 工作流



Technical Focus

01 Character Animation System

角色动画系统

Animation Blueprint · State Machine · Blend Space · Start / Stop / Pivot · Turn in Place · Distance Matching · Root Yaw

02 Procedural Animation

程序化动画

Control Rig · IK · Ground Detection · Foot Placement · Procedural Locomotion · Look-at

03 Gameplay Animation

游戏玩法动画

Motion Warping · Root Motion · Montage · Obstacle Detection · Vault / Mantle / Climb

04 Pipeline / Tools

动画流程与工具

Blender Python

01

角色移动系统 / Character Locomotion System

UE5.6 · Animation Blueprint · Locomotion

问题：

角色转身时，动画网格体和胶囊体朝向不一致，导致：

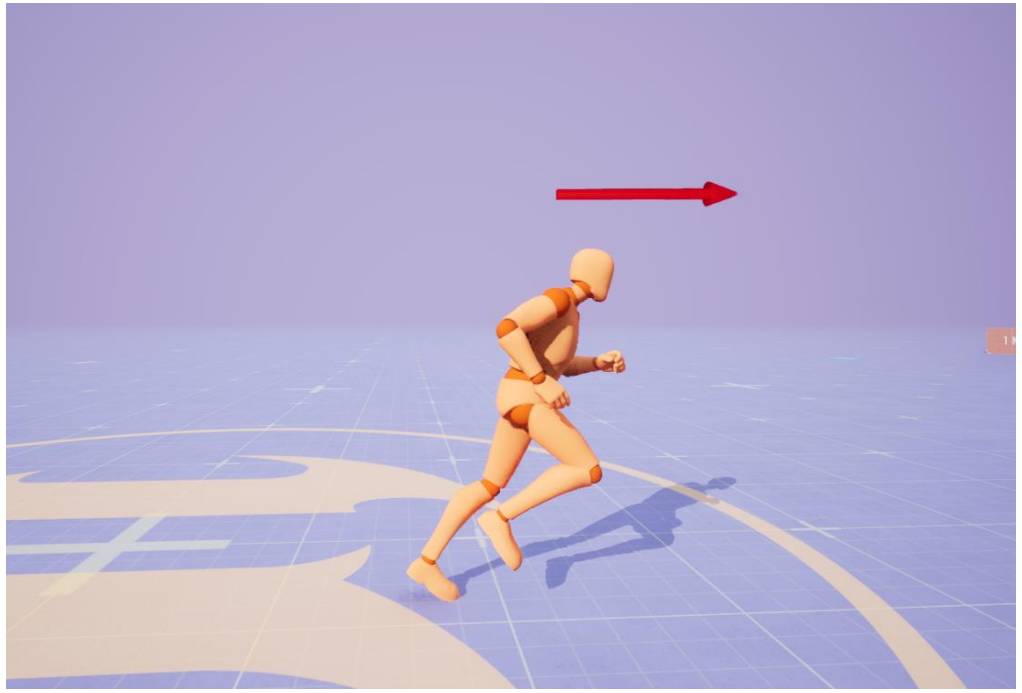
- 脚滑步
- 切换动画时闪动
- 移动中急转没有合适的动画

解决：

基于根偏航对齐的移动动画系统

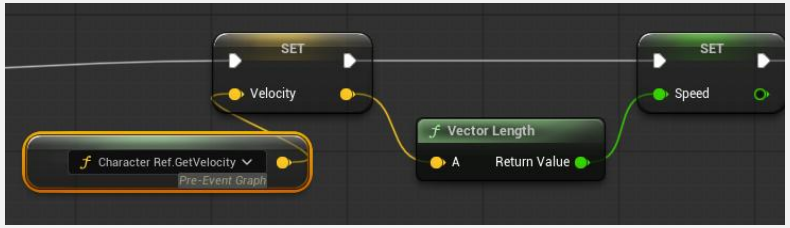
系统覆盖起步、循环、停止、Pivot 与 Turn in Place，并处理方向判断与动画混合。

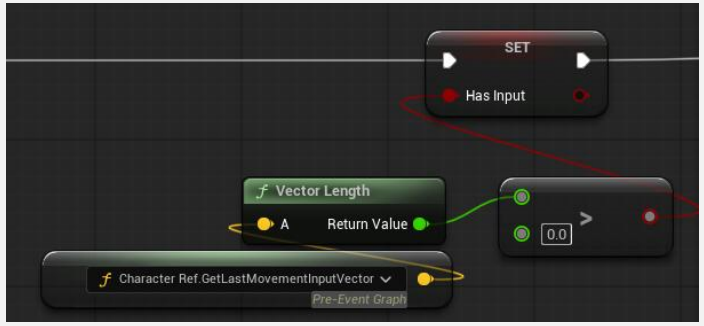
Input → Character Movement → Thread Safe Update → Locomotion Variables → Animation Blueprint → Start / Cycle / Stop / Pivot / TIP

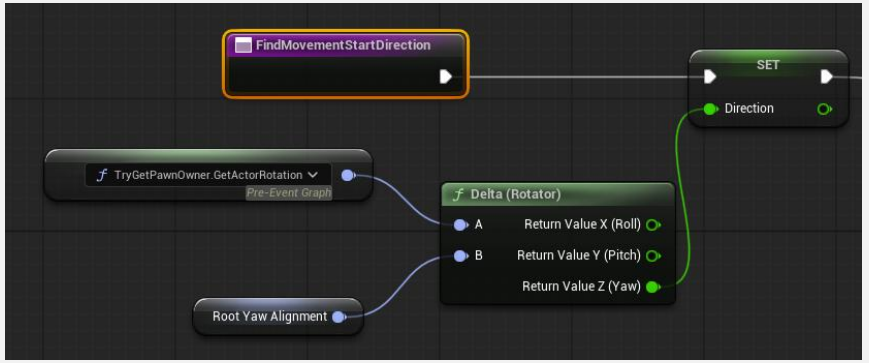


整体架构

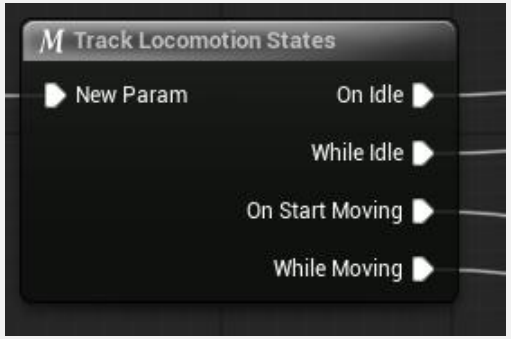
第一层：EventGraph + 函数
　算 Speed、HasInput、IsPivoting、ShouldTIP、MovementStartDirection、RootYawAlignment

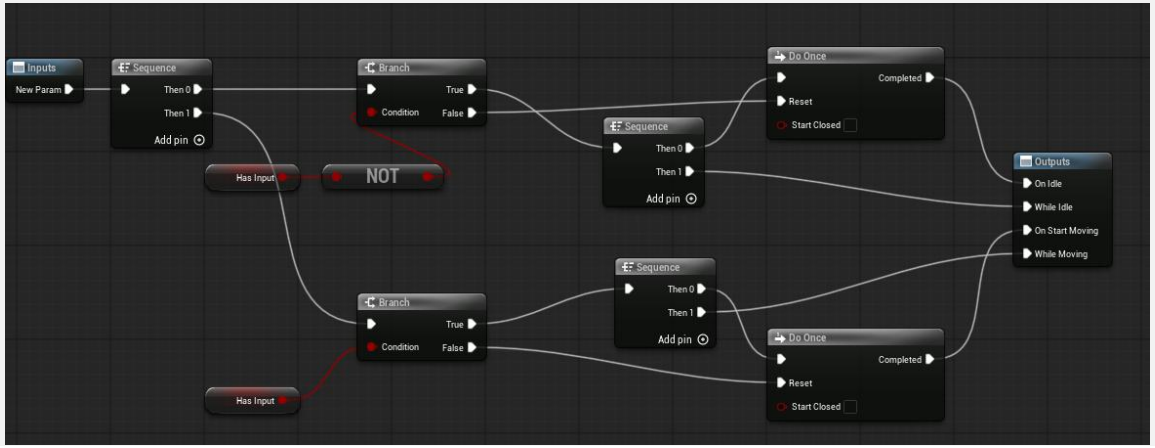






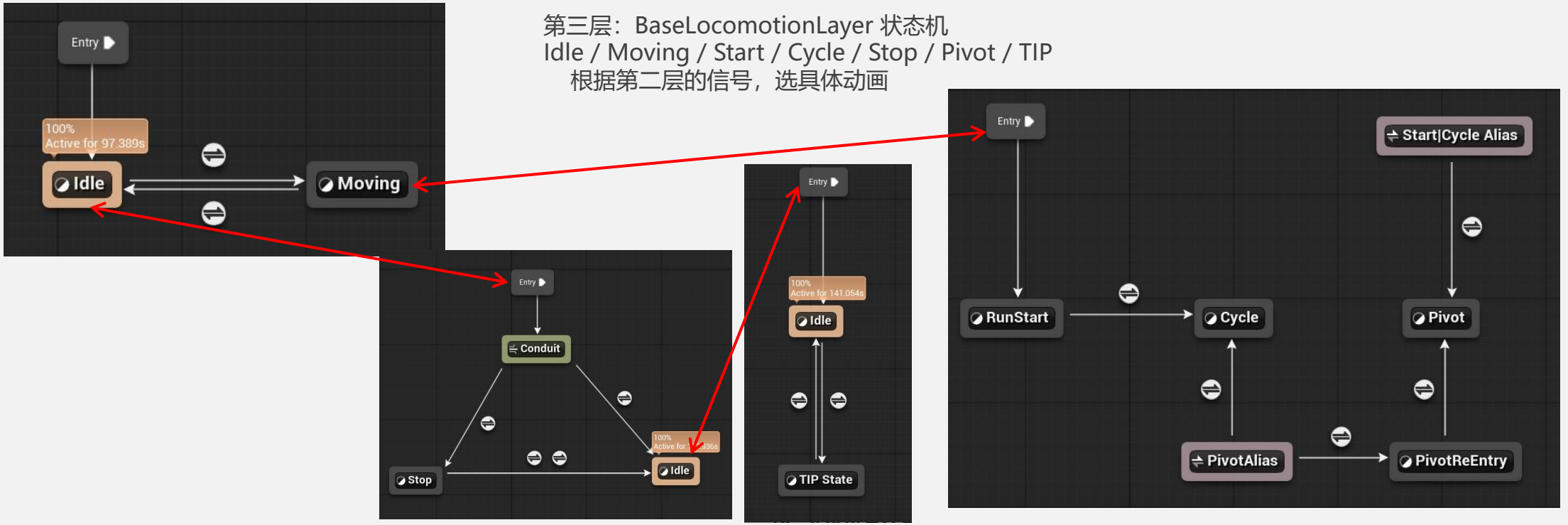
第二层：TrackLocomotionStates 宏
　根据上面这些变量，输出 OnIdle / WhileIdle / OnStartMoving / WhileMoving





整体架构

第三层：BaseLocomotionLayer 状态机
Idle / Moving / Start / Cycle / Stop / Pivot / TIP
根据第二层的信号，选具体动画



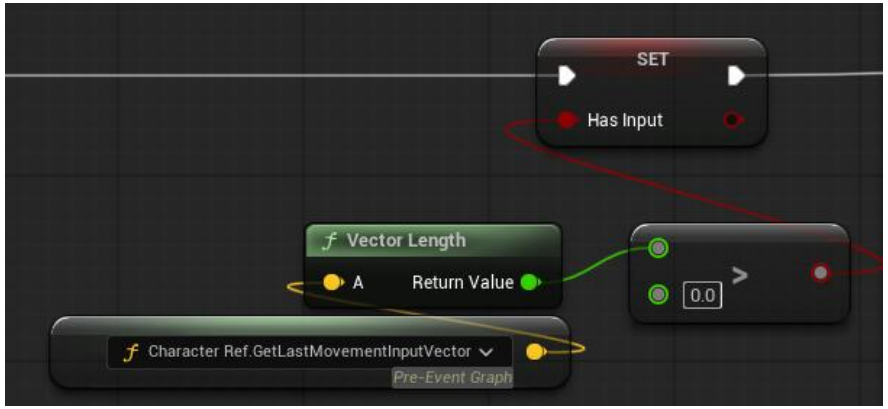
第四层：AnimGraph
DeadBlending -> RotateRootBone -> Root
把选好的动画做根骨骼对齐，最后输出

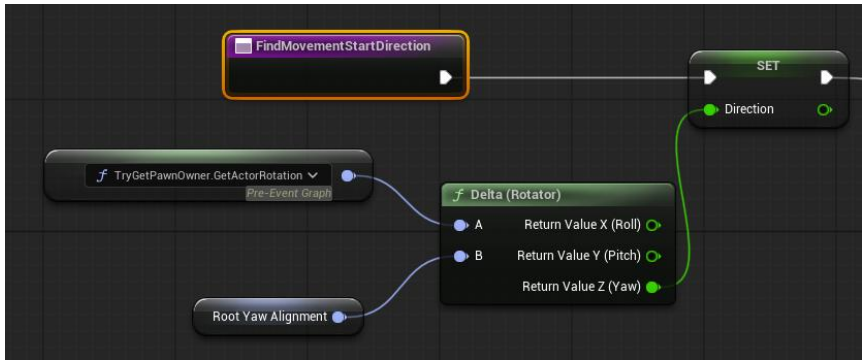


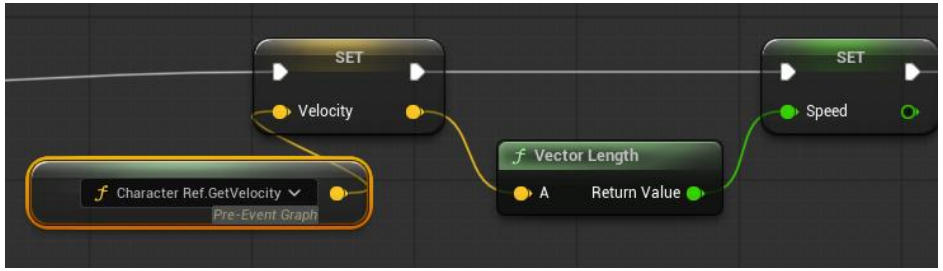
起步系统

起步系统要解决两个问题：玩家有没有输入，往哪个方向输入。用输入向量长度判断有没有输入，用角色朝向和根对齐的差值判断方向，按 60° 分成 6 个扇区，选对应的起步动画。

HasInput = VSize(LastMovementInputVector) > 0
Direction = NormalizedDeltaRotator(ActorRotation, RootYawAlignment).Yaw
MovementStartDirection = 按 60° 分 6 个方向







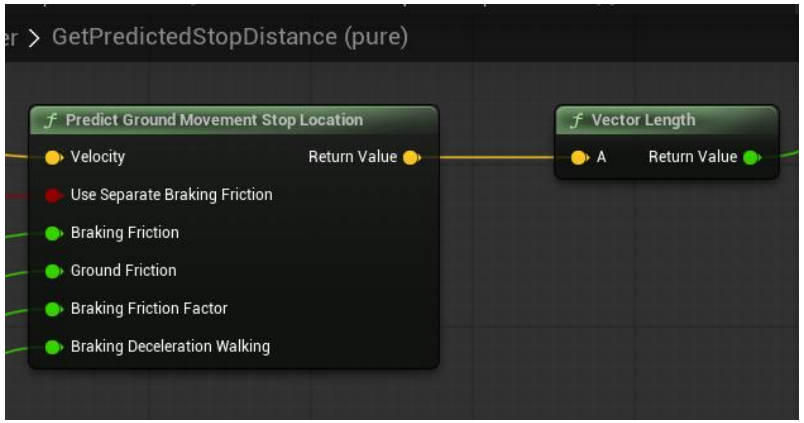
角度范围	枚举
0 ~ +60	Forward Right
0 ~ -60	Forward Left
+60 ~ +120	Right
-60 ~ -120	Left
+120 ~ +180	Backward Right
其余	Backward Left

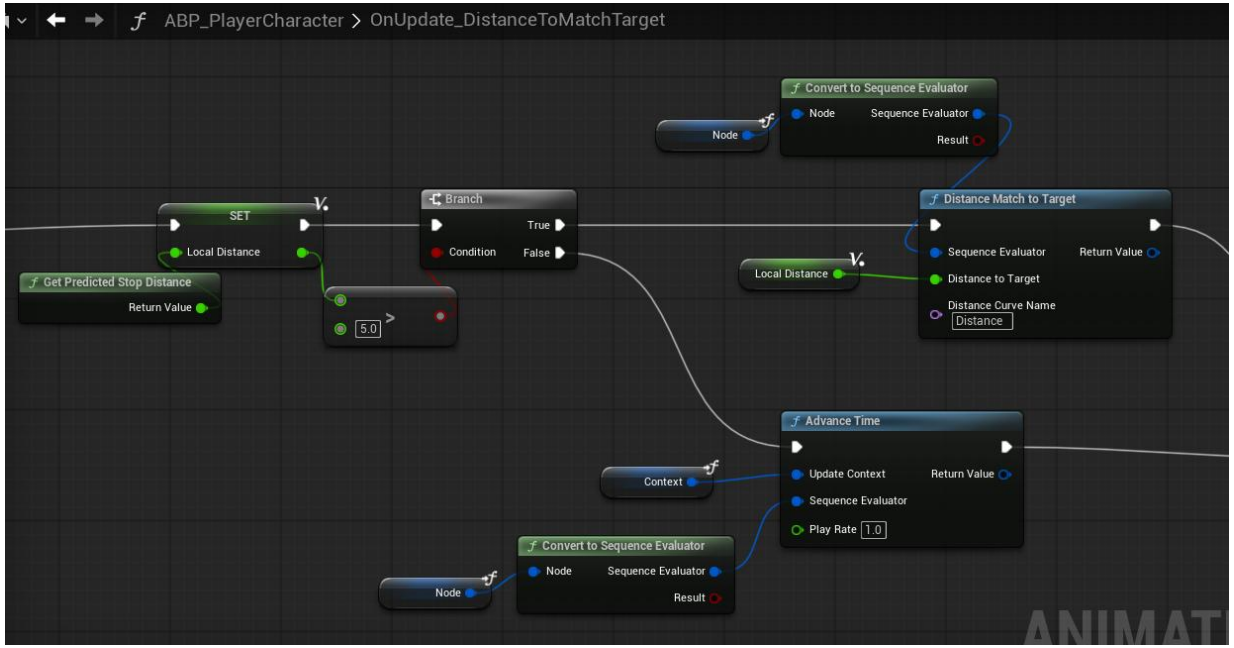
循环 + 停止

循环跑的时候，用脚步同步标记让左右脚对齐，避免脚滑。停下来的时候，先预测角色会停在哪，再反查停止动画该从哪一帧开始播，这样脚正好落在该落的位置。

Predict Ground Movement Stop Location
-> Distance To Match Target
-> Sequence Evaluator 反查停止帧

工具	作用
Distance Matching	先算停点，再决定停止动画从哪一帧播
Foot Sync Marker	标出左右脚落地的帧
Sync Group	让多段动画用同一套脚步标记对齐
Sequence Evaluator	能手动指定播到哪一帧
Inertialization	立即接管，不慢慢混合





Pivot 跑动中急转

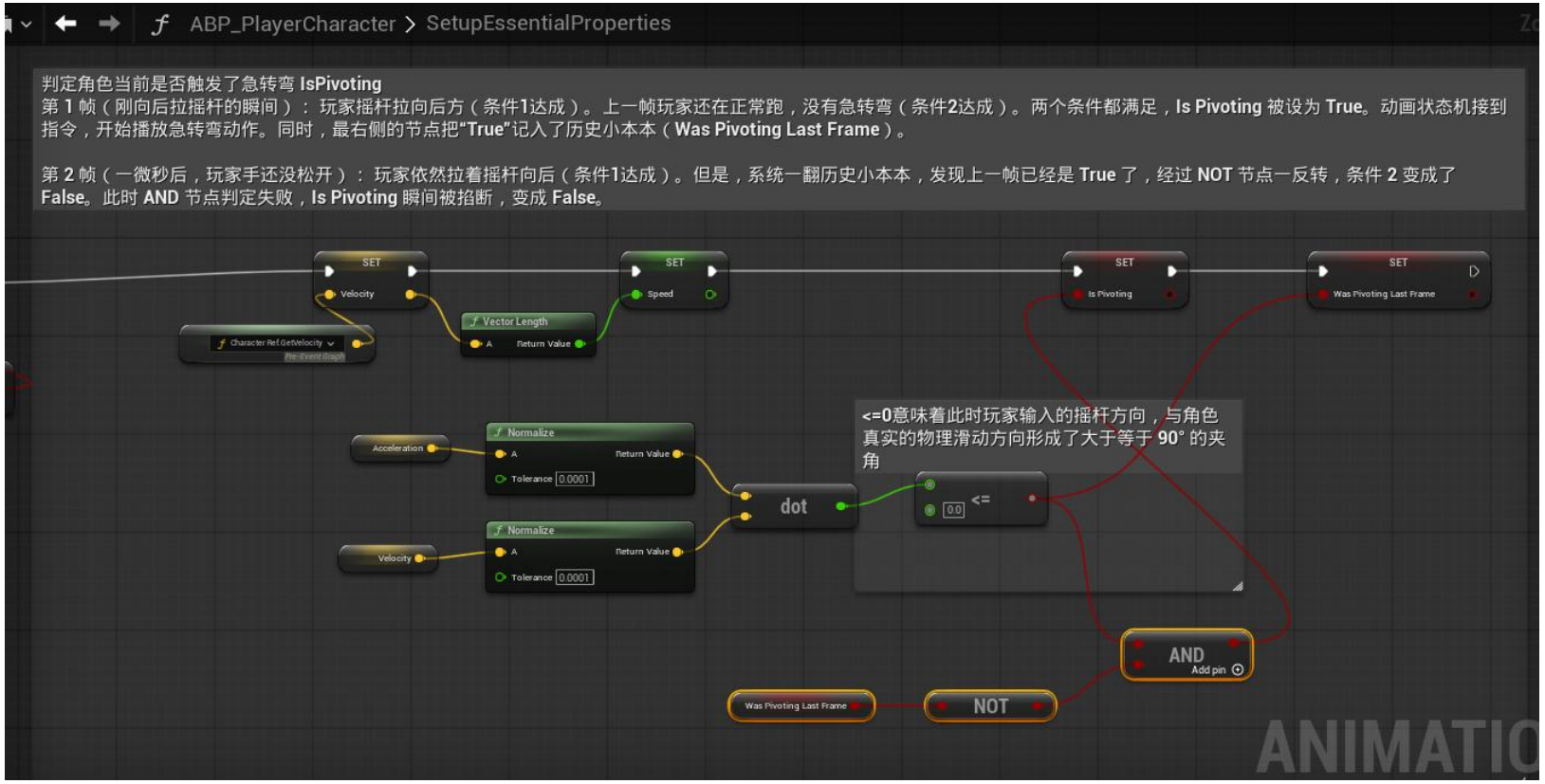
Pivot 检测的是玩家输入方向和角色实际运动方向严重相反的情况。用点积判断，再用边沿触发让它只触发一帧，状态机就能干净地进入 Pivot 状态。

点积 ≤ 0 表示加速度和速度夹角 ≥ 90°，也就是玩家在反向拉摇杆。

Dot(Normal(Acceleration), Normal(Velocity)) <= 0
-> IsPivoting = true

边沿触发让 IsPivoting 只在刚反向的那一帧为 true，避免状态机反复进入 Pivot。

IsPivoting = 当前反向 且 上一帧不是反向
WasPivotingLastFrame = 当前反向

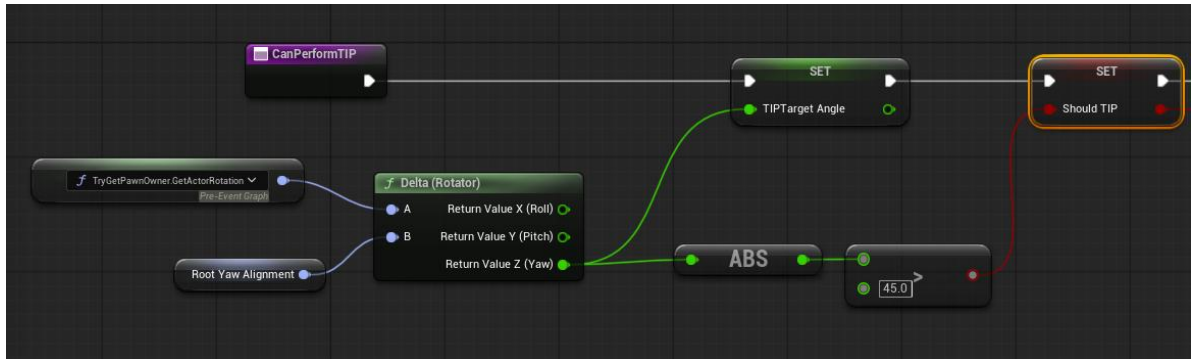


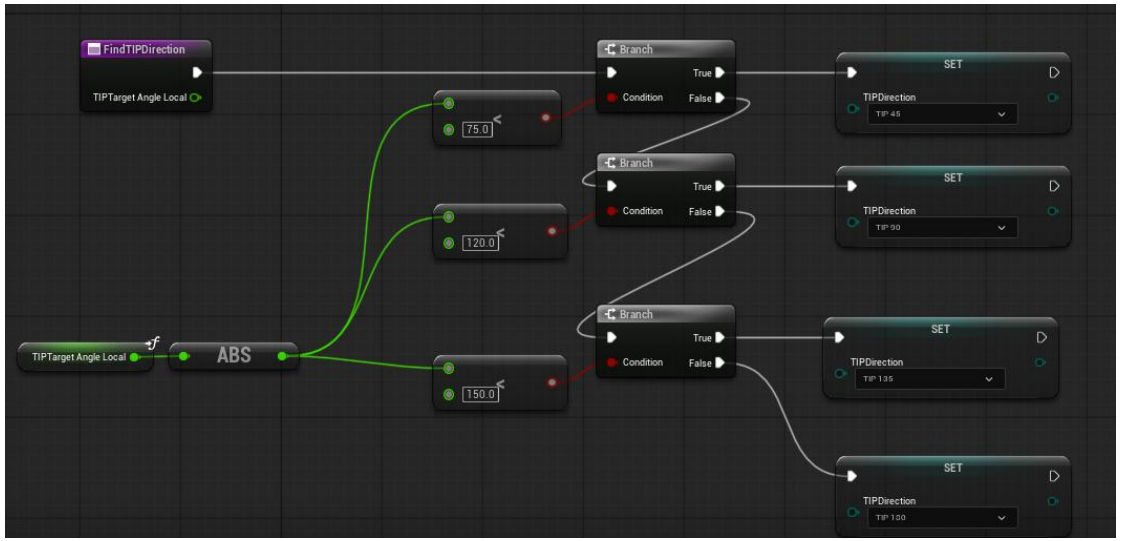
TIP 原地转身

角色停下后，如果控制器转了但脚还没转，就触发原地转身。用角色朝向和根对齐的差值判断差了多少，超过 45° 就转身，再按角度大小选 45/90/135/180 四档动画。

触发逻辑：
Direction = NormalizedDeltaRotator(ActorRotation, RootYawAlignment).Yaw
Abs(Direction) > 45° -> ShouldTIP = true

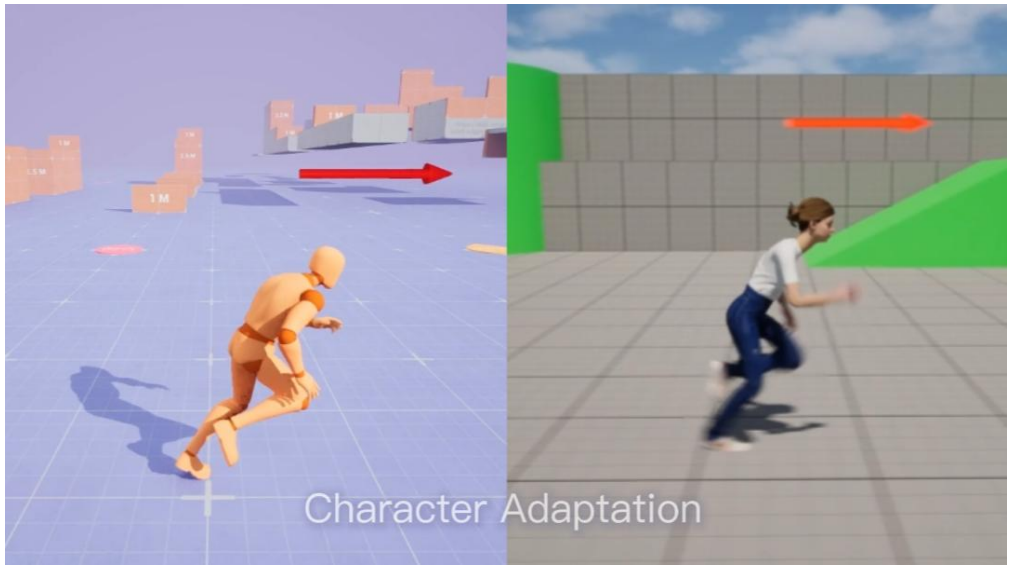
绝对角度	选哪个动画
小于 45	45°
小于 120	90°
小于 150	135°
其余	180°

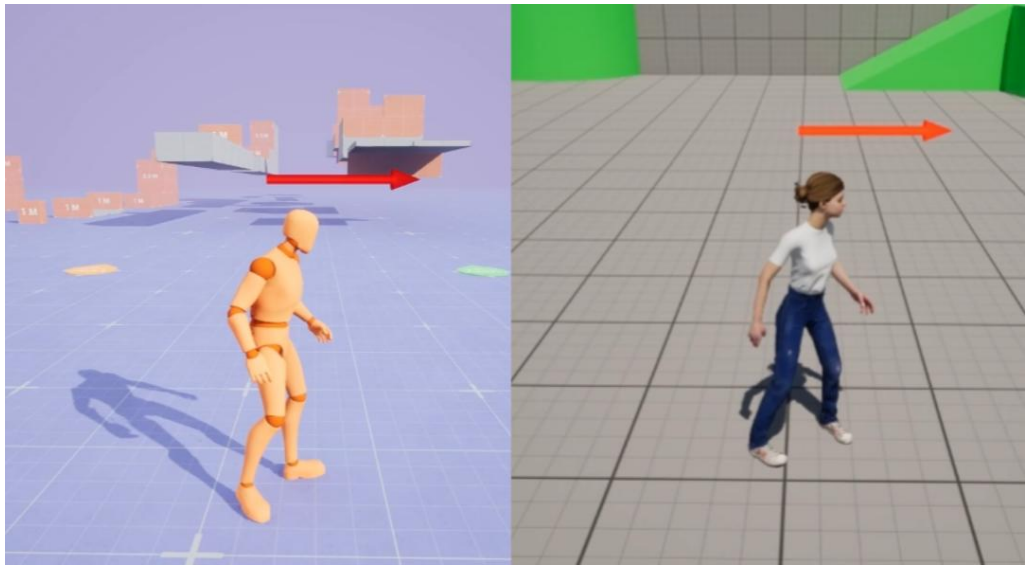




跨角色适配与调试 / Adaptation & Debugging

问题	原因	解决
Mixamo 角色没有 Root 骨骼	Mixamo 骨架默认不带 UE 需要的 root bone	在 UE 骨骼编辑器里手动添加 Root
角色脚滑	一开始以为是重定向问题，排查后发现是 CMC 参数冲突。动画系统和 CMC 同时控制旋转，互相打架	把旋转控制权交给动画系统： CMC Rotation Rate 改成 -1 禁用 CMC 的自动旋转





02

程序化动画 / Procedural Animation

Control Rig · IK · Procedural Locomotion

基于 Unreal Engine 5 实践六足生物程序化行走动画。系统根据角色运动状态计算步态周期，并结合足部预测、地面检测与 Full Body IK 动态生成腿部动作，使角色能够在运动过程中持续调整足部落点。

01 / MOTION

Velocity

根据角色位置变化获取运动状态，为步态系统提供输入。

02 / FOOT PREDICTION

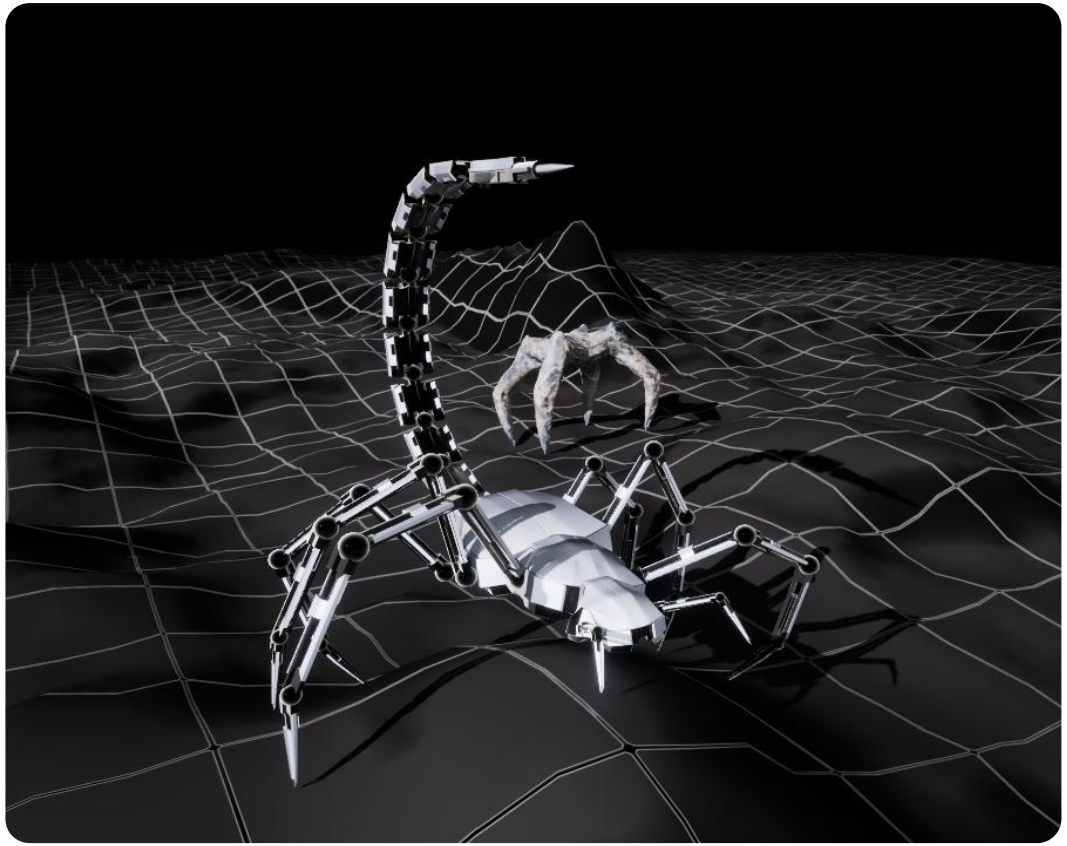
Foot Target

据运动方向预测下一步足部目标位置。

03 / IK SOLVING

Full Body IK

根据目标位置调整腿部骨骼，使足部贴合地面。



六足角色程序化行走/Procedural Creature Locomotion

HOW DOES IT WALK?

Character Motion
↓
Velocity
↓
Cycle / Phase
↓
Foot Prediction
↓
Ground Detection
↓
Foot Target
↓
Full Body IK
↓
Leg / Body Pose

01 VELOCITY 获取角色当前运动状态，为后续步态计算提供基础输入。

02 CYCLE / PHASE 使用步态周期控制六条腿的运动时序，通过相位偏移错开各腿动作。

03 FOOT PREDICTION 根据角色运动方向与步态状态预测下一次足部落点。

04 GROUND DETECTION 使用 Sphere Trace 检测足部目标位置附近的地面。

05 FOOT TARGET 将预测结果与地面检测结果结合，得到最终足部目标位置。

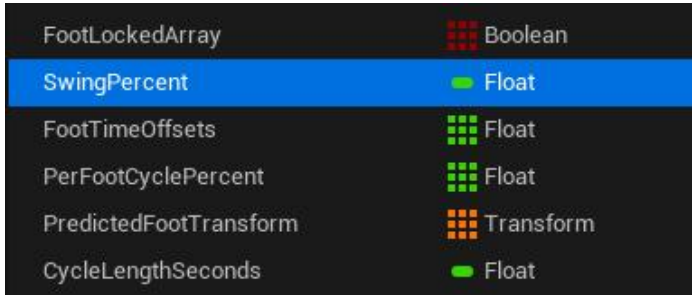
06 FULL BODY IK 将足部目标传递给 IK Solver，调整腿部骨骼以匹配目标位置。

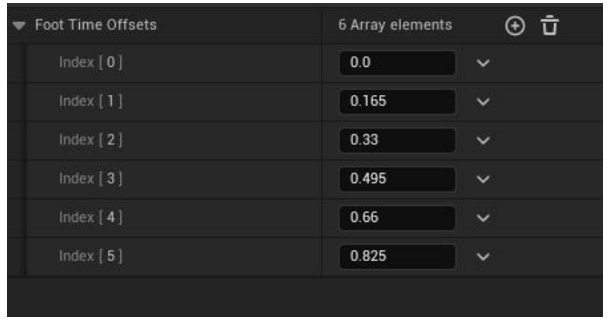
07 BODY ADJUSTMENT 根据腿部姿态变化调整身体姿态，使整体运动保持稳定。

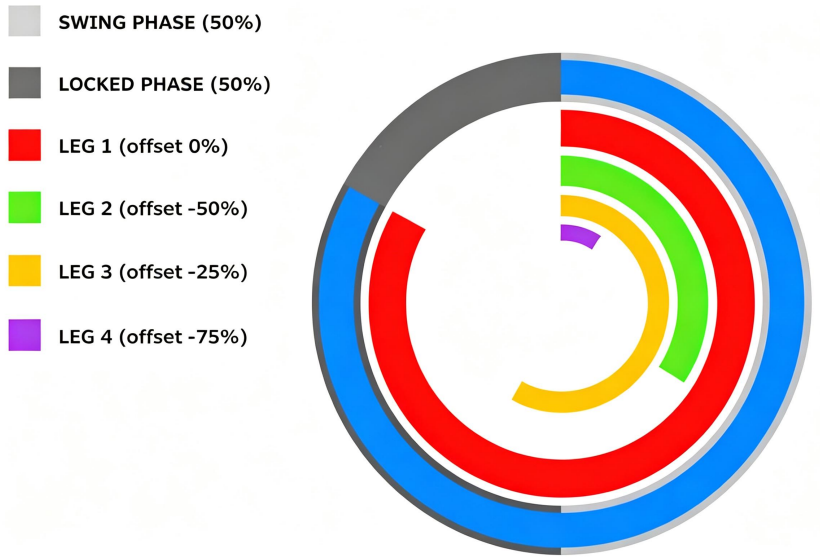


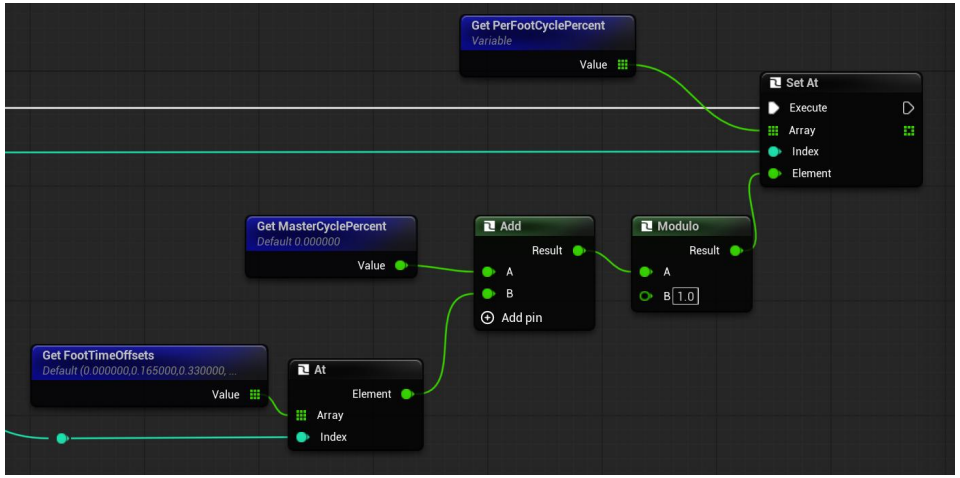
步态周期控制

为了创建自然的行走，引入了一个计时器作为周期。每个周期分为“锁定阶段”（LOCKED PHASE）和“摆动阶段”（SWING PHASE），各占50%。通过错开每只脚的周期时间偏移百分比，实现交替行走的步态。









PerFootCyclePercent[i] = (MasterCyclePercent + FootTimeOffsets[i]) % 1.0

当前脚部周期 = (主周期进度 + 专属相位偏移) % 1.0

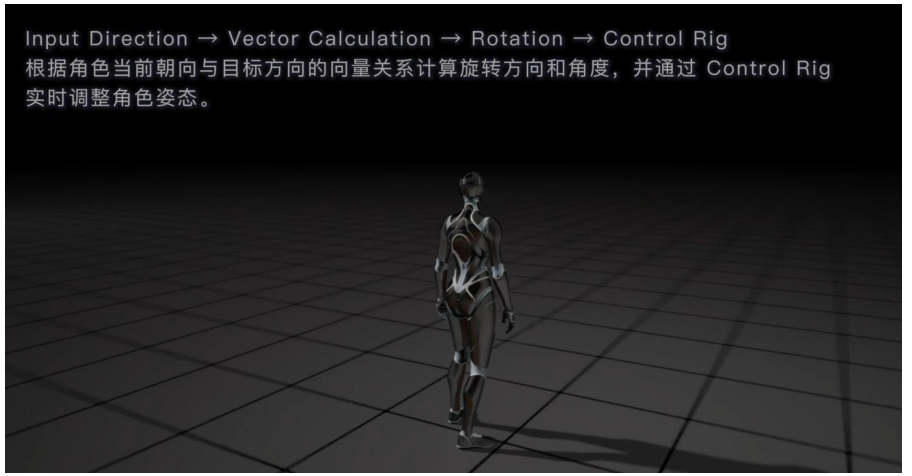
- 遍历角色的六个脚部骨骼，使用全身IK解算器将其移动到指定目标点。
- 创建一个计时器作为周期，决定脚步的锁定与解锁时机。
- 为每只脚分配不同的周期时间偏移百分比，使脚步交替运动。

其他-程序化角色动画实验

Procedural Turn in Place 程序化原地转向

方向驱动转向，摆脱动画依赖。

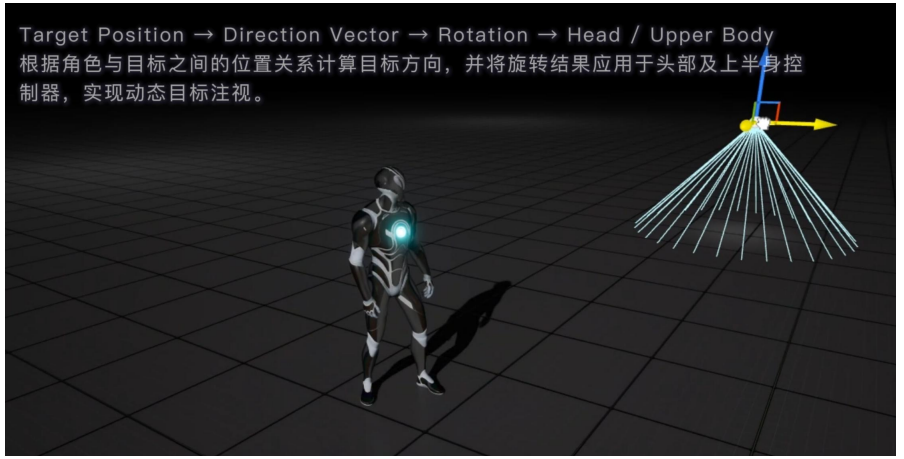
计算朝向差值，经由 Control Rig 实时驱动骨骼旋转，实现平滑的原地位转向。



Look-at / Head Tracking 目标追踪

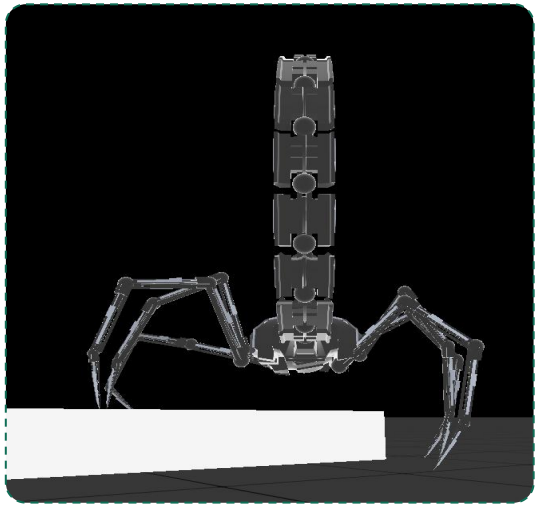
视线锁定，沉浸式交互反馈。

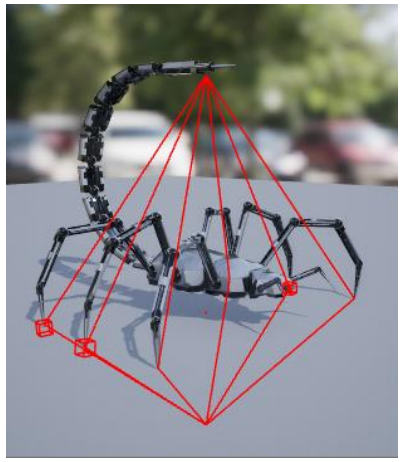
解算目标向量，动态驱动头部与上半身骨骼，实现实时目光锁定与追踪。



地形适配

Sphere Trace → Ground Position → Foot Target





双向球体追踪 → 动态修正落点 → 足部 IK 吸附

通过上下双向 Sphere Trace 实时探测地面法线与高度，将预测的落点(Foot Target)精准吸附至复杂地形，实现多足机器人在台阶、斜坡上的稳定踩踏。

程序化动画重定向



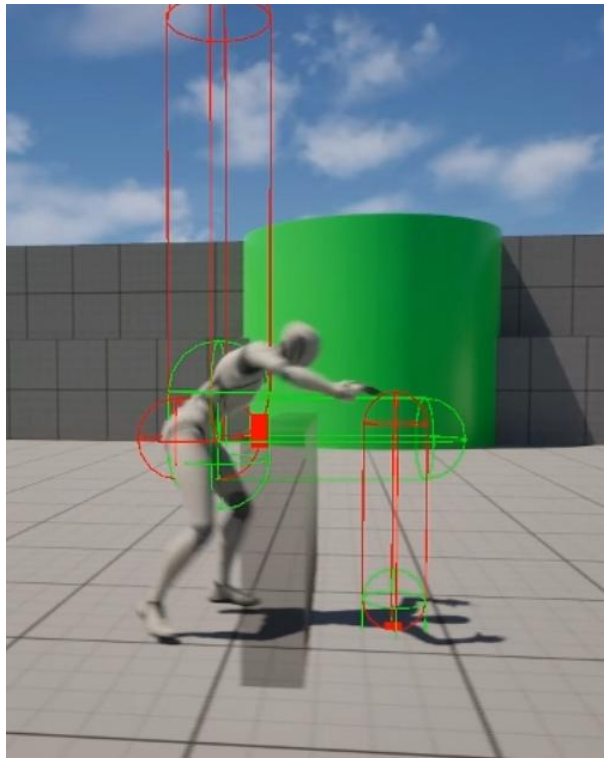
改Rig骨架、删多余数组、重建蓝图，即可用同一套逻辑驱动不同数量腿部的生物（如图所示的四足角色），通过重定向与 IK 解算实现动画资产的高效复用。

04

UE5 跑酷组件 / UE5 Parkour Component

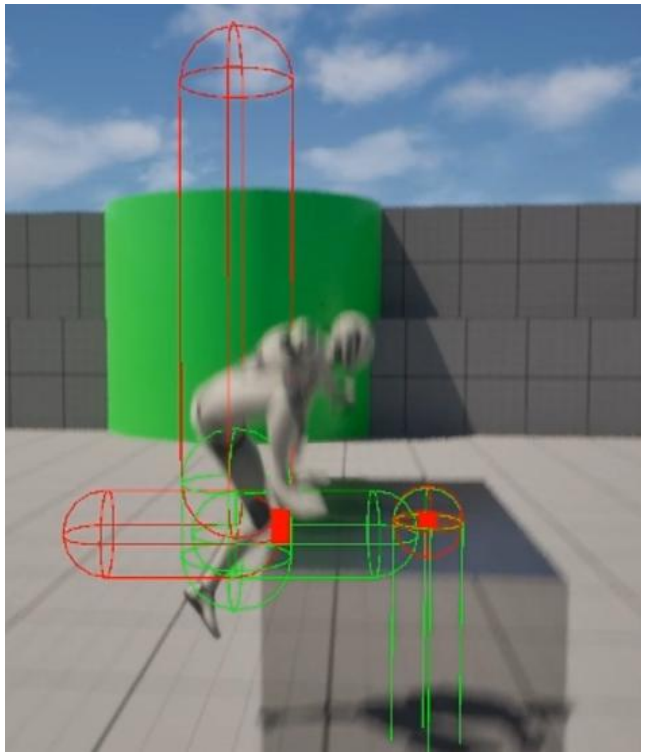
Obstacle Detection

Vault / Mantle / Climb



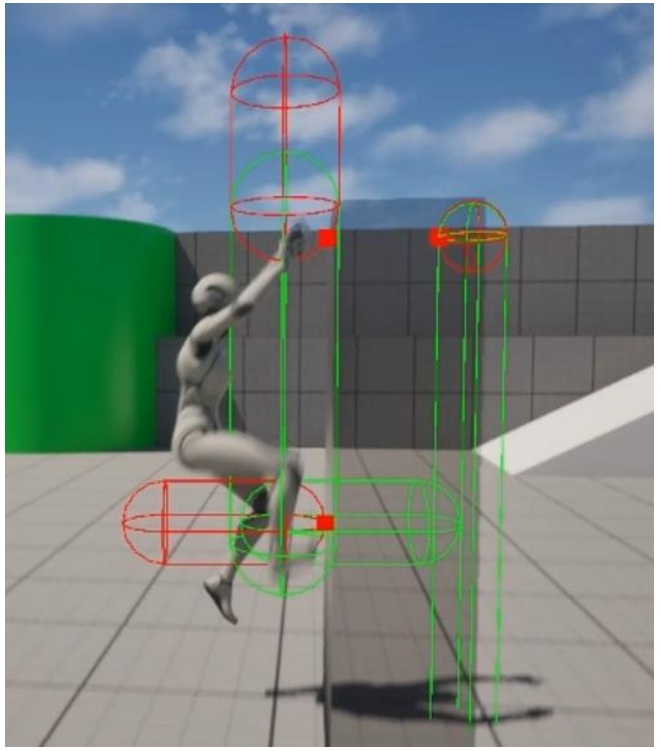
Vault

薄矮墙



Mantle

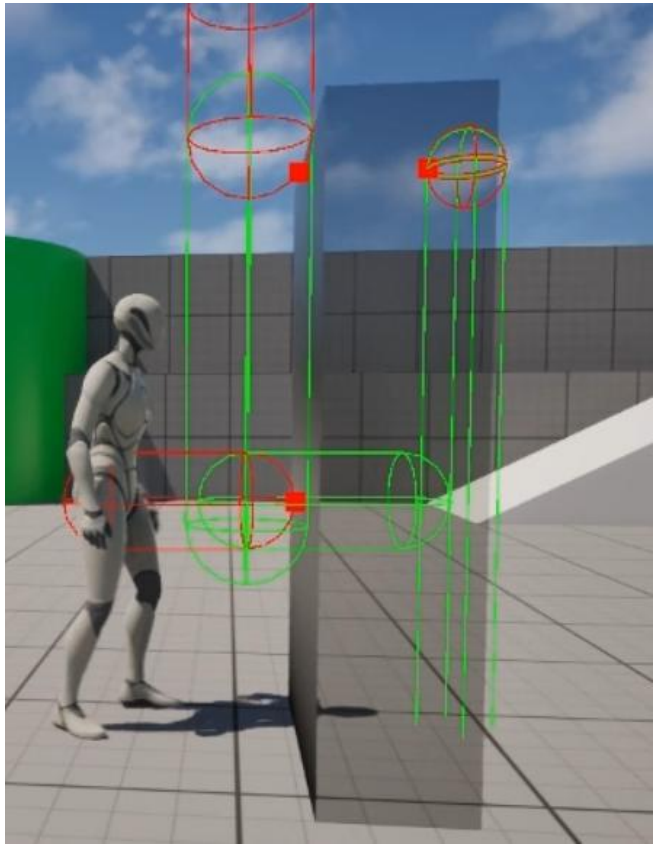
厚矮墙

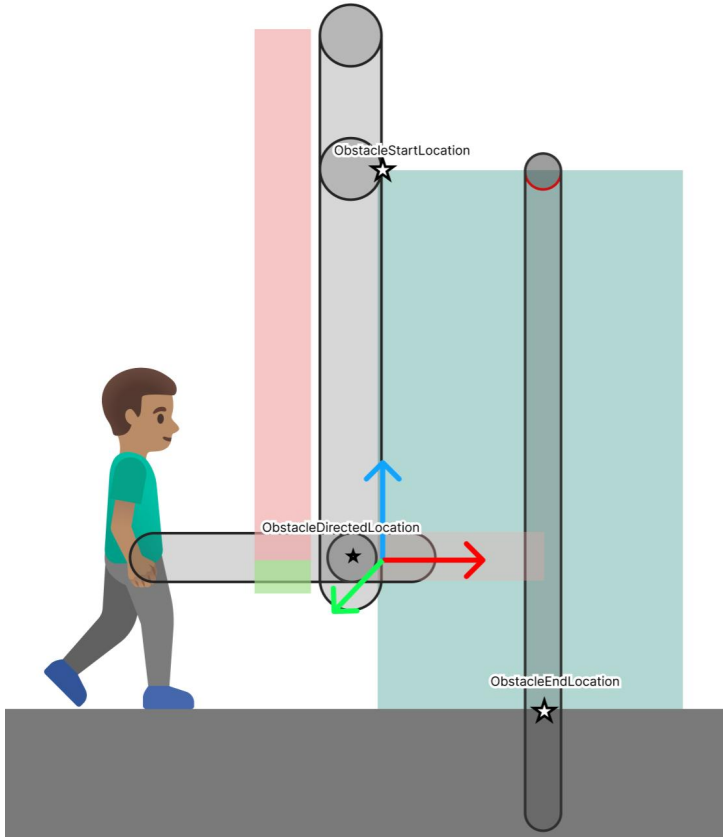


Climb

高墙

多阶段障碍检测+动作判断



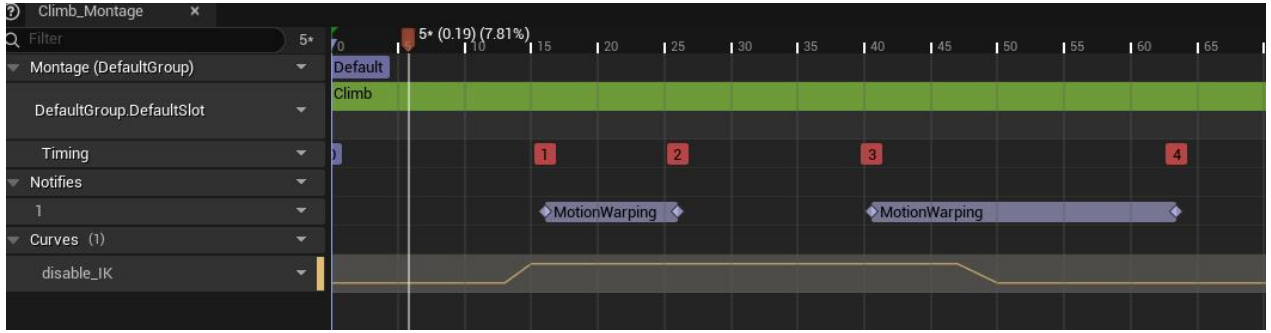


1. IsParkourObstacleValid = false
   → Parkour Action = None（放弃）

2. IsParkourObstacleValid = true
   → 计算 Obstacle Height
   → Obstacle Height > Climb Height ?
   ├─ true → Climb
   └─ false → Is Obstacle Deep ?
   ├─ true → Mantle
   └─ false → Vault

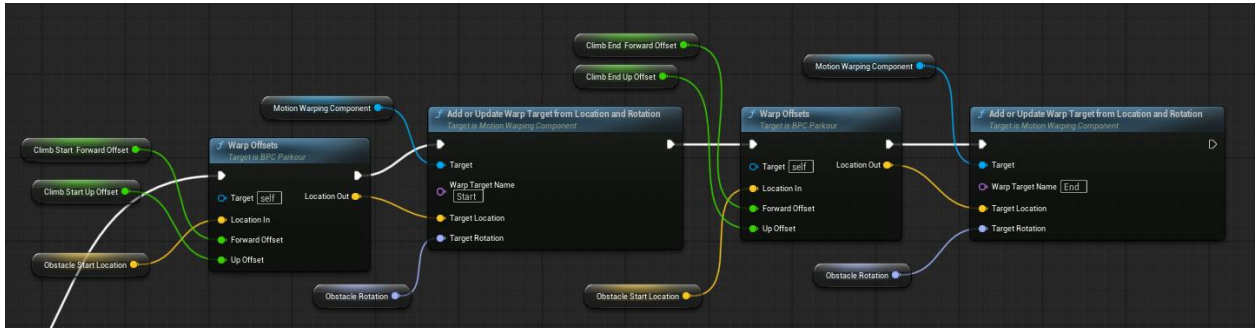
Motion Warping + Foot IK

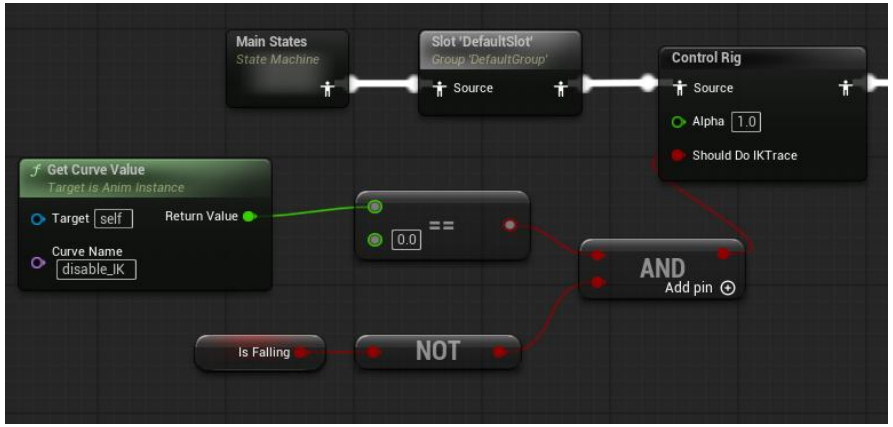
Montage → Motion Warping → Foot IK → Landing



MotionWarping 负责位移对齐——在 Notify 窗口内，按权重把动画原本的 Root Motion 混合成朝目标点的运动，让角色精确吸附到障碍物。

Foot IK 负责脚部贴合——从脚部向下打射线，找到地面后调整脚的姿态。

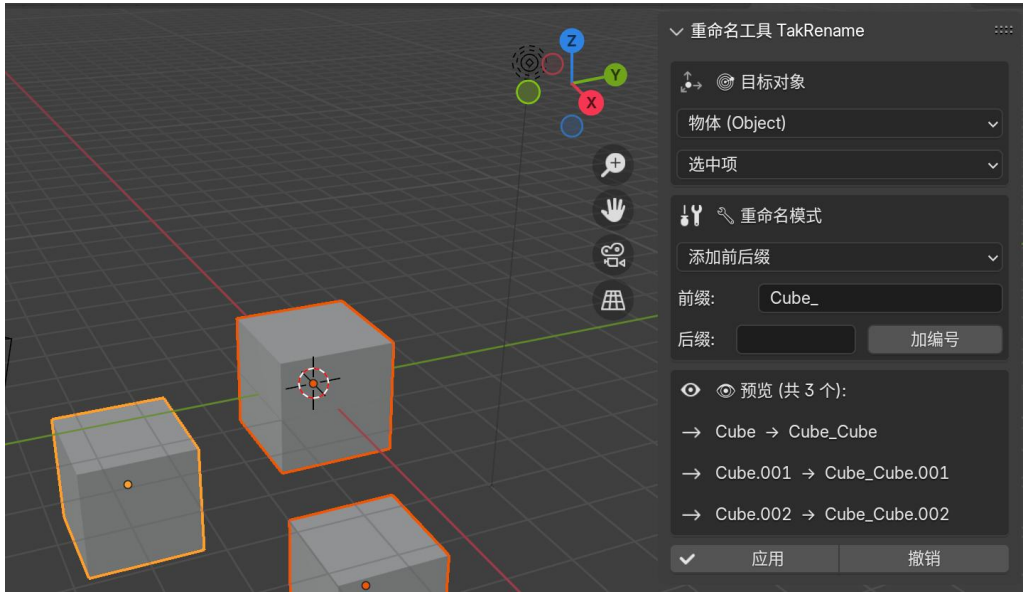




工具 / Tools

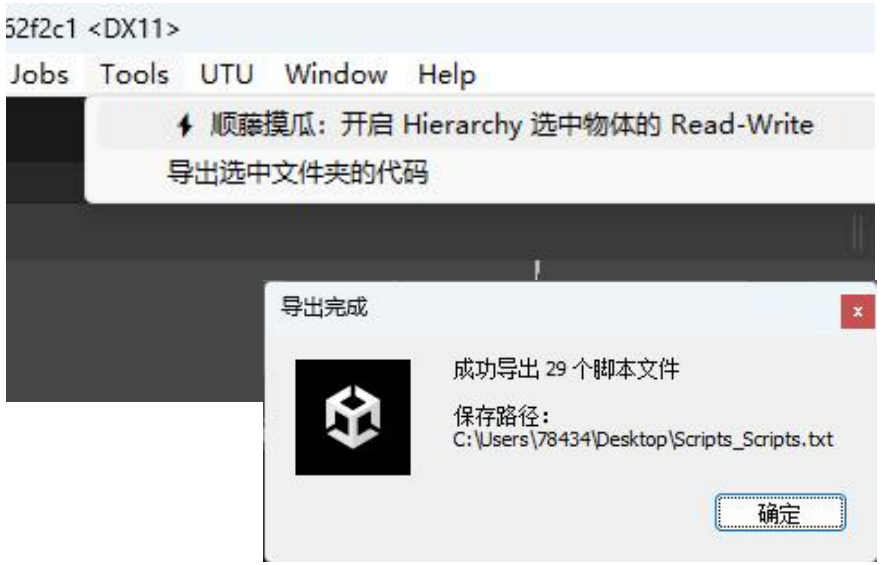
Blender Python

Manual Rename → Python Tool → Batch Rename



Unity Editor

Batch Read/Write → Editor Workflow



其他生产经验 / Additional Production Experience



Blender Animation
Rigging · Skinning · Keyframe Animation · Rendering



MetaHuman
Hair · Groom · Clothing · Character Research

Mail: 784349997@qq.com TEL/Wechat: 13338747156